certora

Security Assessment

Final Report

Jito

Jito Coinbase Integration

March 2026

Prepared for Jito

Table of Contents

Project Summary 3
- Project Scope 3
- Project Overview 3
- Protocol Overview 3

Assessment Methodology 5

Threat and Security Overview 6
- Assets 6
- Actors 6
- Trust Assumptions 7
- Attack Vectors 7
- **Findings Summary** 9
- Severity Matrix 9

Detailed Findings 10
- **Medium Severity Issues** 12
  - **M-01: Hopper is bound only to the whitelist and not the pool, allowing whitelisted members to steal funds from the hopper using their own pool** 12
- Low Severity Issues 13
  - **L-01: Fees conversion from pool tokens to lamports has up to 1 BP of rounding, which can be reduced to 1 lamport** 13
  - **L-02: It's best to use safe casting when converting from u128 to u64** 14
  - **L-03: Whitelisted deposit and withdraw actions lack slippage protection** 15
  - **L-04: Whitelist account creation can be DoS-ed by an attacker sending a few lamports to it** 16
  - **L-05: If check_manager_fee_info() check fails the pool wouldn't charge any fees, but the interceptor would rebate the caller as if it did** 17
  - **L-06: Duplicate checks for admin and whitelist members aren't always effective, allowing the addition of duplicate members** 18
- **Informational Severity Issues** 19
  - **I-01: Removing an invalid entry from the whitelist succeeds silently** 19
  - **I-02: Hopper funds lack a direct recovery path** 20
  - **I-03: Requiring a signature from new admin or whitelist member can help prevent input errors** 21
  - **I-04: admin_info.is_writable is enforced even though the admin account is never modified** 22

Disclaimer 23

About Certora 23

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Fix Commit	Platform
Jito Coinbase Integration	jito-whitelist-management stake-deposit-interceptor	ea7c7df de80e53	9ca381a c120888	Solana

Project Overview

This document describes the security review of **Jito Coinbase Integration**. The work was undertaken from **March 10th, 2026** to **March 19th, 2026**.

The following contract list is included in our scope:

- jito-whitelist-management/core/src/*
- jito-whitelist-management/program/src/*
- stake-deposit-interceptor/stake_deposit_interceptor/src/* (whitelist-related paths and direct dependencies, PR-36 to PR-48)

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Jito is a Solana staking and MEV infrastructure protocol, and JitoSOL is its non-custodial liquid staking token built on the Solana stake-pool program. Users deposit SOL or stake into the JitoSOL pool and receive a liquid token that continues to accrue staking rewards together with MEV-related revenue.

The changes in scope add a dedicated whitelist management program and new whitelisted deposit and withdrawal paths in the stake-deposit interceptor. These paths are intended to

support the Coinbase integration by allowing approved signers to bypass the normal deposit receipt / cooldown flow and to receive withdrawal fee rebates from a pre-funded hopper.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

This section summarizes the threat model for the audited scope, identifying the main assets, actors, trust assumptions, and attack vectors relevant to the whitelisted flows. Because the design introduces whitelist-based access, cooldown bypass, and hopper-funded fee rebates, the main security considerations are access control, configuration integrity, correct binding to the intended Jito stake pool, and safe handling of rebate funds.

Assets

- **JitoSOL supply** handled through whitelisted paths.
- **SOL held in the hopper PDA** - used to fund the atomic withdrawal fee rebate.
- **Stake accounts** entering or leaving the Jito stake pool through the whitelisted path.
- **Protocol fee flows and fee rebates** - especially the withdrawal fee value refunded to whitelisted users.
- **Whitelist membership state** - determines who can access the whitelisted instructions.
- **Whitelist admin control** - determines who can add or remove admins and whitelisted members.
- **Interceptor deposit-authority configuration** - stores the trusted whitelist program, stake pool, pool mint, and stake-pool program.
- Access to **cooldown bypass** and **withdrawal fee-rebates**.

Actors

- **Hopper funder / operator** - tops up the hopper so withdrawal rebates remain available.
- **Whitelist admins** - can manage admins and whitelist entries and therefore control access to the whitelisted paths.
- **Whitelisted signers / Coinbase accounts** - can call the whitelisted instructions.
- **Interceptor authority** - can update the deposit-authority account, including the trusted whitelist program ID and operational parameters.

- **Jito stake pool manager** - manages the pool configs that the whitelisted flow relies on.
- **SPL stake-pool program** - downstream program relied upon for the actual deposit and withdrawal logic and checks.
- **External malicious actors** - may try to front-run initialization, pre-fund the whitelist PDA, or abuse misconfiguration without being whitelisted.

Trust Assumptions

- **Whitelist admins** are assumed to be honest when **adding admins** and **managing whitelist membership**.
- The **interceptor authority** is assumed trusted when setting the **whitelist program** and **related configuration**.
- The **SPL stake-pool program** enforces correctly the downstream **withdrawal** and **stake-account rules** that the interceptor relies on.
- **Whitelisted signers** are expected to act within the **intended integration flow**, but the protocol should still enforce all relevant **safety checks** explicitly.
- **Jito stake pool configuration** remains aligned with the whitelisted flow, including **fee settings and authority relationships**.

Attack Vectors

- **Hopper not bound to the intended pool** - a whitelisted actor may pair it with a different pool and extract hopper funds.
- **Whitelist bypass** - a non-whitelisted signer gains access to the whitelisted instructions.
- **Whitelist/admin slot DoS** - duplicate admin or member entries can consume limited slots and make management unrecoverable.
- **Whitelist initialization lockout** - permissionless singleton initialization or acceptance of an invalid initial admin can break admin control.
- **PDA creation DoS** - attacker can pre-fund the whitelist PDA and block account creation.
- **Fee rebate miscalculation** - arithmetic edge cases, rounding, or mismatch with downstream fee charging can cause the hopper to rebate incorrectly.
- **Hopper draining** - a whitelisted actor repeatedly consumes rebate liquidity, degrading availability for legitimate withdrawals.
- **Stale-rate execution** - whitelisted users have no minimum-out protection.

- **Operational misconfiguration** - pointing the interceptor to the wrong whitelist program, wrong pool, or incorrect admin can break access control or redirect economic benefits.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	1
Low	6	6	6
Informational	4	4	3
Total	11	11	10

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	Hopper is bound only to the whitelist and not the pool, allowing whitelisted members to steal funds from the hopper using their own pool	Medium	Fixed
L-01	Fees conversion from pool tokens to lamports has up to 1 BP of rounding, which can be reduced to 1 lamport	Low	Fixed
L-02	It's best to use safe casting when converting from u128 to u64	Low	Fixed
L-03	Whitelisted deposit and withdraw actions lack slippage protection	Low	Fixed
L-04	Whitelist account creation can be DoS-ed by an attacker sending a few lamports to it	Low	Fixed
L-05	If `check_manager_fee_info()` check fails the pool wouldn't charge any fees, but the interceptor would rebate the caller as if it did	Low	Fixed
L-06	Duplicate checks for admin and whitelist members aren't always effective, allowing the addition of duplicate members	Low	Fixed

I-01	Removing an invalid entry from the whitelist succeeds silently	Informational	Fixed
I-02	Hopper funds lack a direct recovery path	Informational	Fixed
I-03	Requiring a signature from new admin or whitelist member can help prevent input errors	Informational	Acknowledged
I-04	admin_info.is_writable is enforced even though the admin account is never modified	Informational	Fixed

Medium Severity Issues

M-01: Hopper is bound only to the whitelist and not the pool, allowing whitelisted members to steal funds from the hopper using their own pool

Severity: Medium	Impact: High	Likelihood: Low
Files: stake_deposit_interceptor/src/processor.rs	Status: Fixed	

Description:

The hopper's seeds contain only the whitelist address and not the pool, pool program or deposit authority. This binds the hopper only to the whitelist, allowing a whitelisted member to use it with a fake stake pool program or their own instance of the stake pool.

This would allow the attacker to effectively steal the funds from the hopper, since the attacker gets a rebate from the hopper while fees are either not charged or go to the attacker's account as well.

Recommendations:

Bind the hopper to the pool and pool program as well (this can also be done indirectly via the deposit authority, as long as we check that the pool matches the deposit authority as well).

Customer Response:

Fixed in PR #55

Fix Review:

Looks good

Low Severity Issues

L-01: Fees conversion from pool tokens to lamports has up to 1 BP of rounding, which can be reduced to 1 lamport

Severity: Low	Impact: Low	Likelihood: High
Files: stake_deposit_interceptor/src/processor.rs	Status: Fixed	

Description:

Currently the fees' conversion from pool tokens to lamports is done by first getting the conversion rate with basis point precision (i.e. assets*1e4/supply), and then multiplying the fee by the conversion rate and dividing by 1e4.

This can have a precision loss of up to 1 basis point (1e-4 of the total fee). This is while using the pool's calc_lamports_withdraw_amount() function would reduce this only to 1 lamport.

Recommendations:

Switch to using calc_lamports_withdraw_amount() to do the conversion.

Customer Response:

Fixed in PR #56

Fix Review:

Looks good

L-02: It's best to use safe casting when converting from u128 to u64

Severity: Low	Impact: Low	Likelihood: Low
Files: stake_deposit_interceptor/src/processor.rs	Status: Fixed	

Description:

In process_withdraw_stake_whitelisted, the fee rebate calculation performs the intermediate math in u128 but then converts the result back to u64 with unchecked as casts. If either intermediate result ever exceeds u64::MAX, the conversion will silently truncate instead of failing, which can produce an incorrect rebate amount. The conditions required for this seem unlikely under normal pool ratios, but if this edge case is ever reached, the withdrawal would still succeed while paying an incorrect rebate.

Recommendations:

Consider replacing the unchecked as u64 casts with checked conversions such as u64::try_from(...).

Customer Response:

Fixed in PR #57

Fix Review:

Looks good

L-03: Whitelisted deposit and withdraw actions lack slippage protection

Severity: Low	Impact: Low	Likelihood: Low
Files: stake_deposit_interceptor/src/processor.rs	Status: Fixed	

Description:

The whitelisted deposit and withdrawal paths always forward to the plain SPL stake-pool instructions and do not let callers specify any minimum-out constraint, even though the underlying SPL stake-pool supports slippage-protected variants. This also differs from the regular interceptor deposit flow, which already supports minimum_pool_tokens_out. As a result, a whitelisted user can be executed at a worse exchange rate than expected if pool state changes between signing and execution.

Recommendations:

Add an optional minimum_pool_tokens_out parameter to DepositStakeWhitelisted and a minimum_lamports_out parameter to WithdrawStakeWhitelisted, and forward them to the SPL stake-pool instructions.

Customer Response:

Fixed in PR #58

Fix Review:

Looks good

L-04: Whitelist account creation can be DoS-ed by an attacker sending a few lamports to it

Severity: Low	Impact: Medium	Likelihood: Low
Files: program/src/initialize_whitelist.rs	Status: Fixed	

Description:

The CreateAccount system ix reverts if the target account contains any non-zero lamports balance. This would allow an attacker to DoS the creation of the whitelist account by sending to the account as low as the required rent exemption for the account.

Recommendations:

Check the balance, in case it's non-zero create the account by calling assign, transfer (if the balance is below the required rent exemption) and realloc directly.

Customer Response:

Fixed in PR #13

Fix Review:

Looks good

L-05: If `check_manager_fee_info()` check fails the pool wouldn't charge any fees, but the interceptor would rebate the caller as if it did

Severity: Low	Impact: Medium	Likelihood: Low
Files: stake_deposit_interceptor/src/processor.rs	Status: Fixed	

Description:

The interceptor estimates the fees to be zero only if user_pool_token_account_info == stake_pool.manager_fee_account.

However, the pool doesn't charge any fees also if the check_manager_fee_info() check fails. If that ever happens, then the interceptor would rebate the user for fees that they weren't charged.

Recommendations:

Check also check_manager_fee_info() and set the estimated fees to zero if it fails.

Customer Response:

Fixed in PR #59

Fix Review:

Looks good

L-06: Duplicate checks for admin and whitelist members aren't always effective, allowing the addition of duplicate members

Severity: Low	Impact: Medium	Likelihood: Low
Files: core/src/whitelist.rs	Status: Fixed	

Description:

While adding an admin and a whitelist member to the respective lists the function loops over the list and reverts if it finds that the member already exists. However, in the same loop the function also checks if the entry is empty, and if it is it adds the member and returns without checking if the member exists in the rest of the list.

This can lead to a scenario where, for example, the same admin is added multiple times and occupies all 8 slots of the admins list. Making it impossible to add new admins, and the removal of the dupe entries is impossible as well since an admin can't remove himself.

Recommendations:

Check the entire list for duplicates.

Customer Response:

Fixed in PR #14

Fix Review:

Looks good

Informational Severity Issues

I-01: Removing an invalid entry from the whitelist succeeds silently

Files:

jito-whitelist-management/program/src/remove_from_whitelist.rs

Description:

process_remove_from_whitelist() succeeds even when the provided key is not currently present in the whitelist, including when the input is EMPTY_ADDRESS. In contrast, process_remove_admin() validates that the target is an existing admin and reverts otherwise. This silent no-op on incorrect removals can lead to unsuccessful removals being treated as if they had succeeded and state had changed.

Recommendations:

Consider returning an error when attempting to remove a key that is not currently whitelisted, including when the to-be-removed key is EMPTY_ADDRESS.

Customer Response:

Fixed in PR #15

Fix Review:

Looks good

I-02: Hopper funds lack a direct recovery path

Files:

stake_deposit_interceptor/src/state/hopper.rs

Description:

As currently implemented, the Hopper account does not appear to have an authorized recovery path for SOL in case one is ever needed. In practice, funds can only leave the Hopper through fee rebate payouts during whitelisted withdrawals, so excess funds would remain inaccessible unless the program is upgraded. If upgradeability is ever removed, such funds would become effectively non-recoverable and could only be accessed through future fee rebate payments.

Recommendations:

Consider adding a controlled recovery instruction for Hopper funds. Access control could be restricted to the admins of the whitelist associated with the specific Hopper, or to a dedicated recovery authority.

Customer Response:

Withdraw from hopper ix added in PR #53

Fix Review:

Looks good

I-03: Requiring a signature from new admin or whitelist member can help prevent input errors

Files:

jito-whitelist-management/program/src/add_admin.rs

Description:

Requiring a new admin or whitelist member to co-sign the transaction would be an additional on-chain confirmation that the provided address is correct, accessible, and indeed the intended new admin, reducing the risk of accidental/incorrect assignments.
This applies to the initialization ix as well, where the `initial_admin_info` account is assigned as the first admin.

Recommendations:

Consider making the admin/member a co-signer of the tx.

Customer Response:

Acknowledged

I-04: `admin_info.is_writable` is enforced even though the admin account is never modified

Files:

jito-whitelist-management/program/src/add_admin.rs

Description:

In whitelist management program, the admin management instructions require admin_info to be marked writable even though the account is only used for signature and admin-set validation. The admin account is never modified in add_admin, remove_admin, add_to_whitelist, or remove_from_whitelist; only the whitelist account is updated. Besides making the interface stricter than needed, this also unnecessarily reduces parallelism for unrelated transactions using the same signer.

Recommendations:

Remove the admin_info.is_writable checks from the admin management instructions and keep only the signer and admin-membership validation.

Customer Response:

Fixed in PR #16

Fix Review:

Looks good

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.